VLF Health Kit — How-To Guide

This guide explains how to deploy, run, and act on the two scripts in the VLF Health Kit: `01_VLF_Discovery.sql` and `02_VLF_AutoFix_And_Log.sql`. It is intended for DBAs operating a fleet of SQL Server instances. The conceptual background is in the companion `VLF_Wiki.md`.

1. What This Kit Does

The kit answers two operational questions across an arbitrary number of SQL Server instances. First, "which databases on which servers have transaction-log conditions that need attention?" Second, "of those, which can I fix automatically, and which require a human decision?"

`01_VLF_Discovery.sql` is a pure read-only report. It scans every online user database on the instance it is run against and returns a triage table — one row per finding, ranked by severity. Run it ad-hoc or as part of a one-shot fleet sweep.

`02_VLF_AutoFix_And_Log.sql` is the production agent. It scans the same conditions, applies the two safe automatic fixes (correcting bad log autogrowth settings, and issuing a `CHECKPOINT` on `OLDEST_PAGE` waits), and writes every other finding — plus a record of what it fixed — to both the SQL Server error log and the Windows Application event log via `xp_logevent`. Schedule it on a SQL Agent job; integrate the event log entries into whatever monitoring system you already use.

2. Prerequisites

The account running either script must be a member of `sysadmin`, or hold the equivalent of `VIEW SERVER STATE`, `VIEW ANY DEFINITION`, `ALTER ANY DATABASE`, and `EXECUTE` on `xp_logevent`. In practice this means a DBA login or the SQL Agent service account.

The scripts use `sys.dm_db_log_info`, which requires SQL Server 2016 SP2 or SQL Server 2017 and later. On older builds, replace those calls with `DBCC LOGINFO` results captured into a temp table — the rest of the logic is unchanged. The scripts also assume `msdb.dbo.backupset` is intact and not aggressively purged; if your environment trims backup history more often than every 24 hours, raise the lookback window in the "no recent log backup" check accordingly.

`xp_logevent` is enabled by default. If your environment has explicitly disabled extended stored procedures via surface area configuration, re-enable it for the SQL Agent service account or replace the calls with `RAISERROR (..., WITH LOG)`, which has equivalent effect.

3. Files in the Kit

`01_VLF_Discovery.sql` — read-only triage report. Run interactively. Returns a result set of findings plus raw VLF metrics.

`02_VLF_AutoFix_And_Log.sql` — combined scan, autofix, and event-log writer. Designed to run unattended on a schedule.

`VLF_Wiki.md` — reference material on what VLFs are, how they behave, what causes pathologies, and how remediation works conceptually.

This document — operational instructions for running the above.

4. One-Shot Fleet Discovery

To get a fleet-wide picture of VLF health right now, run `01_VLF_Discovery.sql` against every instance and collect the results into a single table. The two practical ways to do this are a Central Management Server (CMS) multi-server query, and PowerShell with the `dbatools` module.

The CMS approach is built into SQL Server Management Studio. Register your instances under a Central Management Server, select the group, and run the script. SSMS will execute it on every registered instance and merge the result sets, prefixing each row with the server name. Export the merged grid to Excel or save it to a SQL table for triage.

The PowerShell approach scales better and integrates with scheduling. The pattern is:

```powershell
$instances = Get-Content C:\ops\sql_instances.txt
$script    = Get-Content -Raw C:\ops\01_VLF_Discovery.sql

$results = foreach ($i in $instances) {
    Invoke-DbaQuery -SqlInstance $i -Query $script -EnableException
}

$results | Export-Csv C:\ops\vlf_findings.csv -NoTypeInformation
```

Either way, the output is a triage list. Sort by severity (`CRIT`, `WARN`, `INFO`) and database, and work through the criticals first.

5. Deploying the Autofix Job

The autofix script is intended to run continuously on each instance. Create a SQL Agent job per instance with a single T-SQL step containing the contents of `02_VLF_AutoFix_And_Log.sql`, owned by `sa` or an equivalent sysadmin login, scheduled every 15 to 60 minutes depending on how active the instance is. A short interval is fine — the script is cheap and idempotent. The two autofixes only fire when the conditions are true, and re-running on a clean instance is a no-op apart from the heartbeat event.

For multi-instance deployment, use whatever configuration management you already have. The dbatools `Install-DbaSqlAgentJob` and related cmdlets work well; a Group Policy or Ansible or

DSC pattern works equally well. The script is self-contained and has no environment-specific configuration aside from the threshold variables at the top.

The first time the job runs on an instance, expect a flurry of `[FIX-1]` events as it corrects bad autogrowth settings that have accumulated for years. Subsequent runs should produce the heartbeat plus only the genuine ongoing issues.

6. Reading the Event Log Output

Every event written by the script is tagged `[VLFCHECK]` followed by a severity bracket — `[INFO]`, `[WARNING]`, or `[ERROR]`. All events use error number 60000, which makes them trivially filterable.

In Windows Event Viewer, open the Application log, filter for source `MSSQLSERVER` and event ID `60000`, and you will see every entry the kit has produced. To narrow further, filter the message text on `VLFCHECK` (which excludes any other 60000-numbered event from coexisting tooling) or on a specific server name.

In SQL Server, the same entries appear in the SQL Server error log. Read them via `sp_readerrorlog` or the SSMS Log File Viewer.

For monitoring integrations, configure your collector — Splunk, Sentinel, Datadog, an SCOM management pack, whatever you use — to ingest event ID 60000 from the Application log on each SQL host, and to alert on `[ERROR]` and optionally `[WARNING]` severities. The `[INFO]` events (autofixes applied, heartbeat) are useful as audit trail but rarely need to wake anyone.

The format of every event is consistent:

```
[VLFCHECK] [SEVERITY] SERVERNAME - [CATEGORY] message. Action: recommendation.
```

`[FIX-1 APPLIED]` and `[FIX-2 APPLIED]` are audit entries — the script changed something and is recording it. `[DBA REVIEW]` is the entry that means "a human needs to look at this." Everything else is bookkeeping.

7. Triage Playbooks

Each `[DBA REVIEW]` event corresponds to one of the following playbooks. Work them in order of severity.

7.1 High VLF Count

Event text: `[DBA REVIEW] High VLF count on [DatabaseName]: vlf_count=N, active=M, log_size_mb=S.`

Decision. Confirm the count via `SELECT COUNT(*) FROM sys.dm_db_log_info(DB_ID())` inside the affected database. If it is genuinely above the warning threshold and the database is

important enough to matter (production, replicated, AG-joined, large), schedule a maintenance window for the rebuild. If the database is a low-traffic system or scratch database, you may choose to leave it.

Procedure during the maintenance window. Take a fresh log backup so the chain is current. Shrink the log to its minimum size with `DBCC SHRINKFILE (logical_log_name, 0)`. The first shrink may not release everything if the active portion is near the end of the file; in that case take another log backup and shrink again. Grow the log back to its target size in deliberate 8 GB chunks using `ALTER DATABASE ... MODIFY FILE (... SIZE = NMB)`. Set autogrowth to a fixed size of 8 GB or whatever is appropriate for the workload. Verify the new VLF count is in the dozens, not thousands.

The full procedure is in section 11 of the wiki.

7.2 LOG_BACKUP Wait

Event text: `[DBA REVIEW] Log truncation blocked on [DatabaseName]: log_reuse_wait_desc=LOG_BACKUP, recovery=FULL.`

Decision. Two questions in order. First: does this database actually require point-in-time recovery, AG membership, log shipping, or replication? If no, the correct fix is to switch to SIMPLE recovery — `ALTER DATABASE [name] SET RECOVERY SIMPLE` followed by a `CHECKPOINT` — and the problem disappears permanently. If yes, the correct fix is to ensure log backups are running.

Procedure if log backups should exist. Check the SQL Agent job history for the log backup job. If it is failing, fix it — common causes are full backup destinations, expired credentials on the backup target, or paths that no longer exist. If it does not exist at all, create one. Take an immediate emergency log backup to unblock truncation: `BACKUP LOG [name] TO DISK = '...'` with `INIT, COMPRESSION`. The log will truncate on completion.

7.3 No Recent Log Backup

Event text: `[DBA REVIEW] No recent log backup on [DatabaseName]: recovery=FULL, last_log_backup=...`.

Decision. This is the same root issue as a `LOG_BACKUP` wait but caught proactively before truncation has actually blocked. Same decision tree applies. If the database genuinely needs FULL recovery, fix the backup job. If not, switch to SIMPLE.

A common discovery here is a database that was put into FULL recovery years ago for a reason no longer remembered, with no log backups ever configured, growing slowly until something noticed. Check with the application owner before changing recovery model.

7.4 ACTIVE_TRANSACTION Wait

Event text: `[DBA REVIEW] Log truncation blocked on [DatabaseName]:`

`log_reuse_wait_desc=ACTIVE_TRANSACTION.`

Procedure. Connect to the affected database and run `DBCC OPENTRAN`. It returns the oldest active transaction's session ID and start time. For richer context, query `sys.dm_tran_active_transactions` joined to `sys.dm_tran_session_transactions`, `sys.dm_exec_sessions`, `sys.dm_exec_requests`, and `sys.dm_exec_sql_text`.

Common findings include a developer session left open in SSMS over lunch, a stuck application connection holding a transaction open behind a network failure, a long-running ETL or reporting job, or a runaway index rebuild. Decide whether to wait, communicate with the session owner, or kill the session with `KILL <session_id>`. Be aware that killing a long-running transaction triggers rollback, which can take as long as the original work — sometimes longer. Plan accordingly.

7.5 REPLICATION / AVAILABILITY_REPLICA / DATABASE_MIRRORING Waits

Event text: `[DBA REVIEW] Log truncation blocked on [DatabaseName]: log_reuse_wait_desc=REPLICATION` (or one of the other variants).

Procedure. The problem is not in the database itself — it is in a downstream consumer of the log. For `REPLICATION`, check the log reader agent (transactional replication) and the CDC capture job. Both surface in SQL Agent and have their own log files. For `AVAILABILITY_REPLICA`, check the AG dashboard for redo lag, secondary connectivity, and synchronization state. For `DATABASE_MIRRORING`, check mirroring partner connectivity and the mirroring queue. In all three cases the fix is at the consumer, not the primary database.

7.6 Log File High Utilization

Event text: `[DBA REVIEW] Log file high utilization on [DatabaseName]: used_pct=N, size_mb=S.`

Procedure. This is an early warning, not necessarily a problem. Check `log_reuse_wait_desc` first — if truncation is blocked, the playbooks above apply. If truncation is fine and the log is simply sized too small for its workload, pre-size it larger. If a one-time event (a huge data load, an index rebuild) is in progress, wait for it to finish.

8. Tuning Thresholds

The default thresholds at the top of `02_VLF_AutoFix_And_Log.sql` are conservative. `VLF_WARN = 200` and `VLF_CRITICAL = 1000` are reasonable starting points but may be too noisy or too quiet for your environment. Raise the warn threshold if you have a fleet of historically neglected databases and need to focus on the worst first; lower it if you are running tight AG topologies where even a few hundred VLFs slow failover. `LOG_USED_PCT_WARN = 80` is a typical capacity threshold and rarely needs changing.

The autofix targets are also tunable. The script sets corrected autogrowth to 1024 MB. For very large databases — multi-terabyte data warehouses, heavy OLTP — 4096 MB or 8192 MB is more appropriate. Adjust the `FILEGROWTH = 1024MB` clause in Phase 2 of the script.

9. Multi-Instance Deployment Patterns

The script is designed to be deployed identically to every instance. The simplest pattern is a SQL Agent job named `Maintenance - VLF Health Check` containing `02_VLF_AutoFix_And_Log.sql` as a single T-SQL step, scheduled every 30 minutes. Deploy it with dbatools:

```powershell
$instances = Get-Content C:\ops\sql_instances.txt
$body      = Get-Content -Raw C:\ops\02_VLF_AutoFix_And_Log.sql

foreach ($i in $instances) {
    New-DbaAgentJob       -SqlInstance $i -Job 'Maintenance - VLF Health Check' -Own
    New-DbaAgentJobStep   -SqlInstance $i -Job 'Maintenance - VLF Health Check' `
                          -StepName 'Run' -Subsystem TransactSql -Command $body
    New-DbaAgentSchedule  -SqlInstance $i -Job 'Maintenance - VLF Health Check' `
                          -Schedule 'Every 30 min' -FrequencyType Daily `
                          -FrequencySubdayType Minutes -FrequencySubdayInterval 30
}
```

For environments using configuration management (Ansible, Chef, DSC), wrap the same logic in your tooling of choice. The script body never changes; only the surrounding job metadata does.

For environments using a centralized monitoring database (a "DBA inventory" pattern), an alternative is to run `01_VLF_Discovery.sql` from a central job against every instance via linked server or PowerShell, and store the results centrally instead of writing to per-instance event logs. Both patterns work; the event-log pattern integrates more naturally with existing SIEM and monitoring stacks.

10. Troubleshooting the Kit Itself

If the SQL Agent job runs but produces no event log output, verify that `xp_logevent` is enabled and that the job step's account has permission to execute it. Confirm the job actually executed by checking `msdb.dbo.sysjobhistory`. If the job throws permission errors on `sys.dm_db_log_info`, the executing account lacks `VIEW DATABASE STATE` on one or more databases — grant it or run as sysadmin.

If `[FIX-1]` events fire repeatedly for the same database and file, something is resetting the autogrowth. Check for a third-party tool, a maintenance plan, or a deployment script that re-

creates the log file with default settings. Fix it at the source rather than letting the autofix fight it.

If the heartbeat event stops appearing, the job has stopped running — check SQL Agent job history and the agent service itself.

If you see `[ERROR] [FIX-1 FAILED]` or `[FIX-2 FAILED]`, the message includes the underlying SQL error. Common causes are databases that are read-only, in restoring state, part of a log shipping standby, or otherwise not in a state that accepts `ALTER DATABASE`. The script catches and logs these without aborting; investigate the specific database.

11. What Success Looks Like

A few weeks after deployment, the steady-state event log on a healthy instance should consist almost entirely of heartbeat events with the occasional `[FIX-1 APPLIED]` when a new database is created or restored. `[DBA REVIEW]` events should be rare and resolved promptly when they do appear. Fleet-wide VLF counts should trend downward as fragmented logs are rebuilt during maintenance windows. Disk-full incidents on log drives should disappear, because the early-warning events catch growth problems before they become emergencies.

That is the goal. The kit handles the boring half automatically and surfaces the interesting half clearly. The DBA's job becomes deciding, not detecting.